

Mini-rapport : vérification des tests du projet

1. Contexte

Le travail a porté sur la préparation de l'environnement Python, le remplissage des trois fiches de cadrage du projet et l'exécution des tests. Au début, certains tests échouaient car les paquets `pandas` et `scikit-learn` n'étaient pas installés. Des tests liés à Git étaient aussi ignorés, car le dépôt se trouve dans le dossier parent de l'application.

2. Cas de cadrage complétés

Les trois cas décrivent des problèmes différents et l'approche envisagée pour les résoudre :

- **Cas 1 – Ordonnances incomplètes en pharmacie** : repérer les informations manquantes sur une ordonnance. L'approche choisie repose sur des règles métier vérifiables, avec une décision finale laissée au pharmacien.
- **Cas 2 – Questions sur le règlement intérieur** : répondre aux questions des employés ou étudiants à partir du règlement. L'approche RAG permet de s'appuyer sur des passages du document et de citer les articles pertinents.
- **Cas 3 – Priorisation des avis négatifs** : classer les avis clients selon leur urgence. L'approche choisie est le machine learning, entraîné à partir d'avis classés, avec vérification par l'équipe.

Chaque fiche précise également l'utilisateur visé, les données nécessaires, un objectif mesurable, les conséquences possibles d'une erreur et les risques de confidentialité.

3. Dépendances Python

Les erreurs d'importation ont été corrigées en installant dans l'environnement virtuel les deux dépendances manquantes :

- `pandas`, pour la manipulation de données ;
- `scikit-learn`, importé dans le code sous le nom `sklearn`, pour les outils de machine learning.

Ces paquets figurent dans `requirements.txt`, avec les autres dépendances prévues pour le projet.

4. Modifications des tests Git

Les tests vérifiaient auparavant la présence du dossier `.git` directement dans le dossier de l'application. La détection a été corrigée dans `tests/test_environment.py` et `tests/test_git_history.py` pour reconnaître également un dépôt situé dans un dossier parent.

5. Vérification et résultats

La suite principale a été exécutée avec la commande :

```
python -m pytest -q tests
```

Les vérifications couvrent notamment le contenu des trois fiches, les dépendances et fichiers attendus, ainsi que l'historique Git et l'absence de fichiers sensibles ou de l'environnement virtuel dans les fichiers suivis. Résultat : **38 tests réussis, aucun échec et aucun test ignoré**.

6. Captures avant et après

Les deux emplacements ci-dessous sont prévus pour ajouter les captures d'écran du terminal. Pour les afficher, placer les fichiers `tests-avant.png` et `tests-apres.png` dans le même dossier que ce document.

Avant les corrections – tests de cadrage uniquement

La commande `pytest tests/test_cadrage.py -q` vérifiait les trois fiches. Le résultat était de **9 tests réussis et 3 échoués**. Les échecs signalaient des rubriques manquantes dans une fiche, une approche non cochée et une justification qui ne citait pas assez de critères.

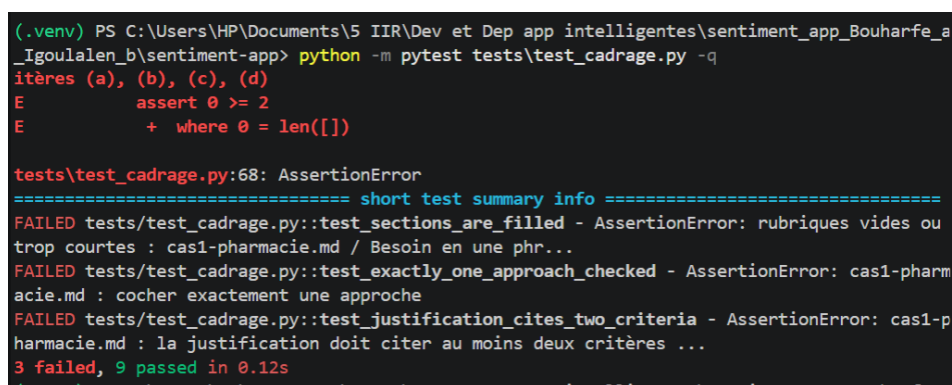


FIGURE 1 – Résultat des tests avant l'installation des dépendances et la correction Git.

Après les corrections – suite principale

Après le remplissage des fiches, l'installation des dépendances manquantes et la correction des tests Git, la commande `python -m pytest -q tests` affiche **38 tests réussis**, sans échec ni test ignoré.

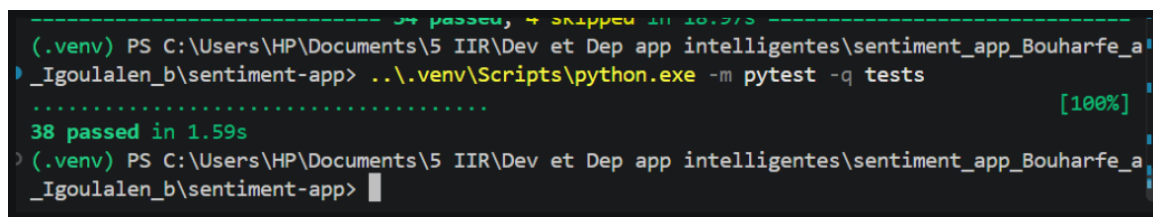


FIGURE 2 – Résultat final des tests après les corrections.

Comparaison

Résultat	Avant	Après
Tests réussis	9	38
Tests échoués	3	0
Périmètre	Cadrage	Suite complète

La capture « avant » concerne uniquement les tests de cadrage, tandis que la capture « après » présente toute la suite principale : le périmètre est donc plus large après les corrections.

Conclusion

Les trois cas de cadrage sont complétés, les dépendances manquantes ont été installées et les tests du projet passent tous. Les contrôles liés à Git, auparavant ignorés, sont désormais exécutés avec succès.